



UNIVERSITÀ DI PISA

Dipartimento di Ingegneria dell'Informazione
Master's Degree in Artificial Intelligence and Data Engineering

Smart Health Monitoring System

Studente: Antonio Pepe
Matricola: 665417

Docenti: Prof. Giuseppe Anastasi
Prof. Carlo Vallati
Ing. Francesca Righetti

Repository GitHub: <https://github.com/apepe11/SmartHealth>

Anno Accademico 2025/2026

Contents

1	Introduzione	2
1.1	Dominio di applicazione	2
2	Schema architetturale	3
2.1	Protocollo di rete	3
2.2	Encoding dei dati	4
2.3	Configurazione dei dispositivi	4
3	Modello di Machine Learning	5
3.1	Funzione principale	5
3.2	Dataset	5
3.3	Addestramento	5
3.3.1	Inferenza sul Dispositivo	6
4	Applicazione Cloud	7
4.1	Backend	7
4.2	Closed-Loop	7
4.3	Interfaccia Utente	8
4.4	Interazioni Hardware	8
4.4.1	Pulsante: Simulazione della Rete Congestionata	8
4.4.2	LED: Indicatori di Stato del Paziente	9
5	Valutazione e stress testing	10
5.1	Scenario Node Failure	10
5.2	Meccanismo Adattivo	10
5.3	Grafana	11
5.3.1	Condizione normale	11
5.3.2	Condizione sotto stress	11

Chapter 1

Introduzione

1.1 Dominio di applicazione

Il dominio di applicazione scelto per questo progetto è stato lo "SmartHealth". Il progetto riguarda un sistema di monitoraggio paziente. L'obiettivo è stato misurare valori come temperatura corporea, battito cardiaco e la saturazione attraverso sensori in modo tale da prevedere situazioni di pericolo o di stato di emergenza così da, attraverso un attuatore, notificare ad un eventuale esperto le condizioni del paziente.

Il mio progetto si è basato su alcuni paper che ho trovato su google scholar:

1. **The Internet of Things for Health Care: A Comprehensive Survey** (*S. M. R. Islam, D. Kwak, M. H. Kabir, M. Hossain and K. S. Kwak — IEEE Access, 2015*)

Fornisce una panoramica sui requisiti di campionamento dei sensori biomedici e formalizza le architetture di rete a tre livelli da Sensore → Gateway → Cloud.

2. **Self-Aware Early Warning Score System for IoT-Based Personalized Healthcare** (*I. Azimi, A. Anzanpour, M. Rahmani-Andebili, et al. — FedCSIS, 2016*)

Questo lavoro descrive la digitalizzazione e l'automazione dei protocolli clinici EWS (*Early Warning Score*) e MEWS all'interno di ecosistemi IoT. Spiega come mappare i dati combinati di temperatura, frequenza cardiaca e saturazione dell'ossigeno (SpO_2) in classi di rischio numeriche. Tale studio costituisce la base logica dell'algoritmo di machine learning.

Chapter 2

Schema architetturale

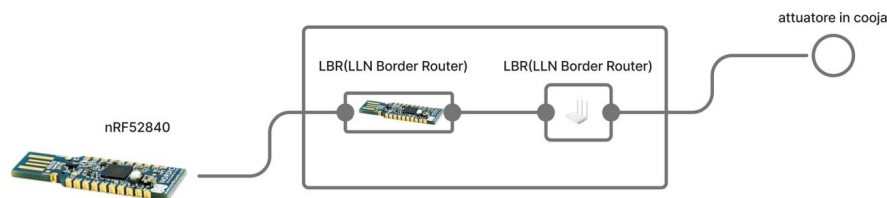


Figure 2.1: Schema architetturale dell'applicazione IoT.

Lo schema dell'applicazione è molto semplice. Abbiamo due LBR (LLN Border Router) collegati attraverso un ponte. La differenza tra i due punti è che uno è stato simulato attraverso Cooja, invece l'altro è fisico e viene simulato attraverso un dongle precisamente l'nRF52840. Inoltre ad ogni LBR è attaccato un dispositivo. Al border router hardware è attaccato un dongle fisico sempre l'nRF52840, invece al LBR simulato via Cooja si attacca un attuatore simulato.

2.1 Protocollo di rete

Come protocollo di rete ho deciso di utilizzare CoAP (Constrained Application Protocol). E' un protocollo a livello applicazione basato su UDP, quindi automaticamente è più leggero, ma come punto a suo sfavore ha che non è affidabile. In realtà per la situazione in cui ci troviamo quindi progettare reti per IoT, quindi dispositivi con risorse limitate, questo protocollo è perfetto. Inoltre il protocollo mette a disposizione vari metodi e varie funzioni, infatti abbiamo che il protocollo è valido sia lato client che lato server. Infine CoAP, per definizione, è un protocollo molto leggero, quindi ottimo per reti di tipo LLNs (Low Power and Lossy Networks).

2.2 Encoding dei dati

La modalità di encoding che ho utilizzato è stata JSON (JavaScript Object Notation). Ho optato per questa soluzione perchè JSON è uno dei formati più comuni utilizzati, inoltre non ha tutte le regole che invece ha un file XML (Extended Markup Language). I file JSON inoltre sono file semplici, avendo una sintassi ristretta per gli oggetti e gli array. Nel mio progetto ,come si può vedere nel file `res_vitals.c` :

```
int length = snprintf((char *)buffer, preferred_size,
    "{\"hr\": %d, \"body_temperature\": %d, \"spo2\": %d, \"risk\": %d}\",
    current_hr, current_temp, current_spo2, current_risk);
```

Listing 2.1: Costruzione del payload JSON in `res_vitals.c`

Si capisce che il payload è davvero di piccole dimensioni. Tutte queste motivazioni si sommano per avere una maggiore efficienza energetica per i dispositivi IoT.

2.3 Configurazione dei dispositivi

La configurazione dei dispositivi è stata la parte relativamente più complessa, a causa della scelta di configurazione hardware e software. Ho configurato i nodi come spiegato nella sezione che parla del sistema architetturale. La configurazione dei nodi è stata fatta in modo statico attraverso un file python (`configuration_manager.py`):

```
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "1",
    "database": "SmartHealthIoT"
}

SENSORS_CONFIG = {
    "Sensori": {
        "id": 1,
        "sensor_ip": "fd00::f6ce:36b9:a760:ecea", # IP reale del
            Dongle Fisico
        "actuator_ip": "fd00::202:2:2:2" # attuatore in Cooja
    }
}
```

Listing 2.2: Configurazione del database e dei sensori in `configuration_manager.py`

La scelta della configurazione statica era da requisito di progetto. Anche nel file `SmartHealthCloud.py`, ritroviamo la configuraizone dei nodi. Come si può vedere in questo stralcio di codice oltre la configurazione statica dei dispositivi c'è anche la configurazione delle credenziali per il DB.

Chapter 3

Modello di Machine Learning

3.1 Funzione principale

E' stato usato un modello di Machine Learning, più precisamente il Random Forest per generare, partendo da un dataset, più alberi decisionali, in base ai parametri vitali. Il modello ha collezionato dati riguardanti la temperatura, battito cardiaco e l'ossigenazione. Grazie a questi tre parametri sono riuscito ad addestrare il modello ed associare ad ogni situazione una classe di rischio. Come da progetto, si è fatto uso del tinyML per portare il modello sul sensore (che ha risorse limitate).

3.2 Dataset

Il dataset utilizzato per l'addestramento del modello di Machine Learning è stato reperito su Kaggle. Nello specifico, è stato utilizzato il seguente dataset:

<https://www.kaggle.com/datasets/jacobhealth/health-status-dataset>

Il dataset descrive la relazione tra temperatura corporea, frequenza cardiaca e saturazione dell'ossigeno nel sangue (SpO_2). Lo stato di salute del paziente è rappresentato da un'etichetta numerica che va da 0 a 2, secondo la seguente classificazione:

- 0 – Nessun parametro fuori dalla norma rilevato;
- 1 – Il paziente è malato, ma la condizione non è in pericolo di vita;
- 2 – Il paziente è in pericolo di vita.

Nel dataset troviamo più di 5000 tuple.

3.3 Addestramento

L'addestramento del modello è stato effettuato utilizzando Google Colab, come documentato nel notebook `ML_model.ipynb` che si trova nella directory `Machine_Learning/` del progetto.

La procedura ha seguito cinque passi fondamentali.

1. **Primo passo:** caricamento del dataset e aggiornamento librerie.
2. **Secondo passo:** preprocessing. Le tre feature disponibili (temperatura, frequenza cardiaca e SpO₂) sono state selezionate come input del modello, mentre la colonna target è stata utilizzata come etichetta per la classificazione.
3. **Terzo passo:** scelta del modello. È stato scelto un algoritmo Random Forest, un classificatore che combina multiple decision tree per ottenere predizioni affidabili. La scelta è ricaduta su questo algoritmo perché è particolarmente adatto all'esecuzione su dispositivi embedded con risorse limitate, grazie alla possibilità di essere convertito in codice C efficiente tramite la libreria emlearn.
4. **Quarto passo:** addestramento e validazione attraverso Google Colab.
5. **Quinto passo:** esportazione per il deployment embedded. Il modello addestrato è stato convertito in codice C tramite la libreria emlearn. Il file generato, `vitals_classifier.h`, contiene la funzione `vitals_rf_model_predict()` che viene poi inclusa direttamente nel firmware del nodo sensore.

```
# Divisione accademica: 80% Training per l'albero, 20% Test per la
validazione
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# 3. ADDESTRAMENTO RANDOM FOREST
# (Ottimizzato per Contiki-NG)
print("\nAddestramento del modello Random Forest sui dati reali di
Kaggle...")
# Mantengo 5 alberi e profondita' 3 per non saturare
# la memoria RAM limitata dei nodi in Cooja
random_forest_model = RandomForestClassifier(
    n_estimators=5, max_depth=3, random_state=42
)
random_forest_model.fit(X_train, y_train)

# 4. VALUTAZIONE DELL'ACCURATEZZA
y_pred = random_forest_model.predict(X_test)
print("\n--- REPORT DI ACCURATEZZA TINYML ---")
print(classification_report(y_test, y_pred))
print(f"Gradi di rischio elaborati dal modello: "
      f"{random_forest_model.classes_}")
```

Listing 3.1: Addestramento del modello Random Forest in `ML_model.ipynb`

3.3.1 Inferenza sul Dispositivo

Il file `vitals_classifier.h` viene incluso nel codice del nodo sensore, precisamente nel file `sensor.c`. A ogni ciclo di campionamento, il sensore esegue localmente l'inferenza del modello. I tre parametri vitali vengono inseriti in un array di feature e passati alla funzione di predizione, che restituisce il livello di rischio clinico. Questo valore viene poi esposto tramite la risorsa CoAP `/vitals` e inviato al backend cloud.

Chapter 4

Applicazione Cloud

4.1 Backend

Per quanto riguarda il backend, ma anche il frontend, tutto il sistema è stato scritto in Python. Ho preferito utilizzare un ambiente virtuale (venv) per isolare tutte le dipendenze. Il sistema cloud si trova nella cartella `Cloud_Application/`. All'interno di questa cartella troviamo due file che riguardano l'applicazione lato backend:

- **db_setup.sql**: Questo script SQL crea il database `SmartHealthIoT` e la tabella `Health_Measurements`, definendo le colonne per memorizzare i dati vitali (frequenza cardiaca, temperatura, SpO_2 , rischio clinico, stato del sensore e timestamp) provenienti dai nodi IoT. Include inoltre un indice per velocizzare le query sui dati più recenti e istruzioni per aggiornare la struttura della tabella senza perdere i dati esistenti, adattandola a eventuali versioni precedenti del sistema.
- **SmartHealthCloud.py**: Questo backend Python usa il meccanismo *Observe* di CoAP per ricevere in modo asincrono i dati dai sensori senza doverli interrogare periodicamente. Quando arrivano i dati JSON, fa il parsing, salva le misurazioni nel database MySQL e, se il livello di rischio clinico è cambiato, invia un comando all'attuatore per aggiornare lo stato dell'allarme (verde, giallo, rosso).

4.2 Closed-Loop

Il sistema che ho sviluppato funziona in questo modo: il sensore raccoglie dati inerenti ai parametri vitali citati precedentemente (in realtà alcuni parametri vengono generati in modo randomico, come anche alla temperatura viene aggiunto un gap perché il sensore simula la sua temperatura interna). Successivamente, il sensore utilizza il modello Random Forest per fare inferenza e determinare, dai dati raccolti, la classe di rischio clinico, realizzando così un approccio di *edge computing*. In seguito, il sensore invia la classe di rischio e i dati vitali tramite il protocollo CoAP.

Il sistema di backend, implementato nel file `SmartHealthCloud.py`, riceve i dati via CoAP *Observe*, li salva nel database MySQL e implementa la logica decisionale che determina se e come agire sull'attuatore.

Il nodo simulato in Cooja espone la risorsa CoAP `/threshold` e reagisce ai comandi ricevuti dal backend, modificando il colore del LED in Cooja in base al livello di rischio ricevuto.

4.3 Interfaccia Utente

L'interfaccia utente è stata sviluppata attraverso la libreria Python Tkinter. Per avviare l'interfaccia è necessario attivare l'ambiente venv e spostarsi nella directory `Cloud_Application/frontend/` ed eseguire il comando:

```
python SmartHealthUI.py
```

Fatto questo, si aprirà un pannello dove vengono visualizzati tutti i parametri attualmente misurati dal sensore e la classe di rischio inferita direttamente dal nodo sensore. Inoltre, è possibile controllare il *rate* di aggiornamento. L'utente può regolare la frequenza con cui la dashboard interroga il database, adattandola alle proprie esigenze, come per esempio simulare una rete trafficata.

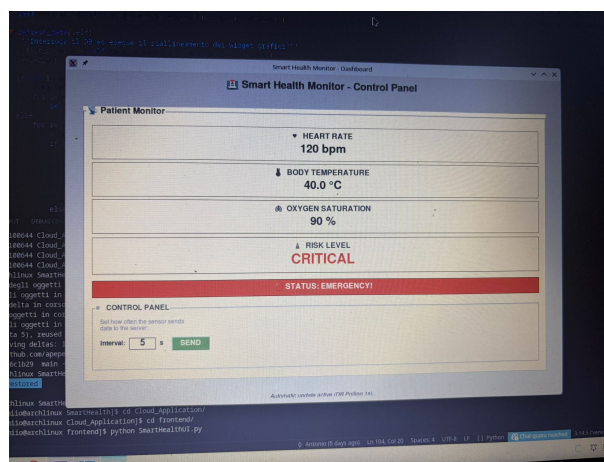


Figure 4.1: Screenshot interfaccia UI per SmartHealth

4.4 Interazioni Hardware

Il sistema SmartHealth utilizza i pulsanti e i LED disponibili sui nodi IoT, sia quelli fisici sul dongle NRF52840 sia quelli simulati nell'ambiente Cooja.

4.4.1 Pulsante: Simulazione della Rete Congestionata

Nel file `sensor.c`, il pulsante hardware è stato configurato per simulare una condizione di rete congestionata, permettendo di attivare la modalità di *stress test*. La logica è implementata nel processo principale del sensore:

```
if(ev == button_hal_press_event) {
    is_network_congested = !is_network_congested;

    if(is_network_congested) {
        LOG_INFO("STRESS TEST: Rete congestionata rilevata!
        Attivazione Meccanismo Adattivo...\n");
        current_sampling_rate = 20;
        etimer_set(&periodic_timer, current_sampling_rate *
        CLOCK_SECOND);
    }
}
```

```

    } else {
        LOG_INFO("Rete tornata stabile. Disattivazione Meccanismo
            Adattivo.\n");
        current_sampling_rate = 5;
        etimer_set(&periodic_timer, current_sampling_rate *
            CLOCK_SECOND);
    }
}

```

Premendo il pulsante, il sistema commuta lo stato della variabile `is_network_congested`. Quando la rete viene segnalata come congestionata, il *sampling rate* viene aumentato da 5 a 20 secondi, simulando il comportamento adattivo che il sistema adotterebbe in una reale condizione di traffico elevato. Premendo nuovamente il pulsante, si torna alla condizione normale.

4.4.2 LED: Indicatori di Stato del Paziente

Nel progetto vengono utilizzati anche i LED. Non avendo a disposizione più dongle fisici, l'attuatore è stato simulato attraverso Cooja. L'interazione con il LED avviene tramite l'interfaccia grafica del simulatore.

L'attuatore, che espone la risorsa CoAP `/threshold`, reagisce ai comandi inviati dal backend modificando il colore del LED in base alla classe di rischio:

- **Rischio 0 (normale):** LED verde, situazione stabile.
- **Rischio 1 (attenzione):** LED giallo, il paziente è malato ma non in pericolo di vita.
- **Rischio 2 (emergenza):** LED rosso, il paziente è in pericolo di vita.

Chapter 5

Valutazione e stress testing

5.1 Scenario Node Failure

Il sensore reale, collegato via USB al PC e flashato con il firmware `sensor.c`, può essere fisicamente scollegato. Quando questo accade, il nodo scompare dalla rete RPL e il backend cloud non riceve più notifiche CoAP. Il sistema se ne accorge perché il meccanismo *Observe* di CoAP smette di ricevere risposte, e il backend stampa un messaggio di errore:

```
[ ERRORE DI RETE] Impossibile raggiungere Sensore_1
```

Anche nell'interfaccia utente viene effettuato un controllo: quando il sensore non è raggiungibile, tutti gli elementi della dashboard vengono disabilitati.

A livello di database, la tabella `Health_Measurements` include una colonna `status` che può assumere il valore `ONLINE` o `NODE_FAILURE`. Quando un sensore smette di inviare dati per un certo periodo, il sistema aggiorna questo campo per segnalare il guasto, permettendo alla dashboard di mostrare lo stato del nodo e a un operatore di intervenire.

5.2 Meccanismo Adattivo

La perdita di pacchetti e i ritardi di comunicazione sono stati simulati attraverso il meccanismo adattivo implementato nel sensore stesso, attivabile tramite la pressione del pulsante hardware.

Come mostrato nel file `sensor.c`, quando l'utente preme il pulsante, il sistema commuta la variabile `is_network_congested` e modifica il *sampling rate*:

```
if(is_network_congested) {  
    current_sampling_rate = 20; // rallenta a 20 secondi  
} else {  
    current_sampling_rate = 5;  // torna a 5 secondi  
}
```

Questo simula il comportamento che il sistema adotterebbe in una rete realmente congestionata.

5.3 Grafana

5.3.1 Condizione normale

In condizioni normali, il sensore campiona e invia i dati ogni 5 secondi. I grafici mostrano un flusso regolare e continuo: i punti sono ravvicinati e costanti, la classe di rischio è stabile, il sensore risulta ONLINE. È il comportamento base del sistema, quello che ci si aspetta in una rete non congestionata.

5.3.2 Condizione sotto stress

Quando si preme il pulsante sul sensore, il meccanismo adattivo si attiva e il sampling rate passa da 5 a 20 secondi. Sulla dashboard questo cambiamento è visibile immediatamente: i punti sul grafico si diradano, perché i dati arrivano con meno frequenza. La cosa importante è che i dati continuano ad arrivare correttamente: il sistema non perde pacchetti, semplicemente rallenta per non sovraccaricare la rete.